The Team Reference Document & a Full WF Simulation

25 pages that never lie, and 5 hours rehearsed end to end

Learning goals

- Plan a **25-page TRD**: what goes in, what stays out, page budget
- Lay it out for **fast lookup and error-free typing**
- **Test every snippet** and keep the document reproducible (hashes, CI)
- Run a **5-hour World Finals simulation** with a written plan
- Debrief with a **post-mortem template** that produces action items

Where we are

- WF-01: format, archetypes, 5-hour strategy — today we *execute* it
- WF-02 … WF-09: the algorithms that must be in the TRD
- Nationals-01: notebook basics, training system
- After today: monthly full simulations until the Finals

> The TRD is not a library you read during the contest. It is a set of **pre-verified** keystrokes.

The rules (check the current year!)

- ICPC WF: one printed **team reference document**, up to **25 pages**, brought to the contest
- No electronic copies; you retype code from paper
- Typical format restrictions: page size, single-sided, minimum font size — read the rules page for your year
- Regionals vary: some allow more pages, some forbid it
- Consequence: **every line costs typing time** — prefer short, robust code

What belongs in a TRD

Include a snippet if **all** are true:

1. It appears in WF-level problems (seen ≥ 2 times in recent sets)
2. You cannot write it bug-free from memory in < 10 min
3. It is **tested** and you have **used it** in practice

Exclude: BFS, Dijkstra, DSU, basic segment tree (you know them), textbook theory, anything nobody on the team has typed before.

Page budget (25 pages)

Section	Pages	Contents
Setup	0.5	template, compile/stress/hash scripts
Math & number theory	2	CRT, Miller–Rabin, Pollard, Lucy, Du
Polynomials	2.5	NTT, FFT, inv/log/exp/sqrt, BM, interpolation
Linear algebra & LP	1.5	Gauss (mod p, real), det, XOR basis, simplex
Graphs & matching	4	Dinic, MCMF, Hopcroft–Karp, Hungarian, blossom, 2-SAT, BCC, global min cut
Trees	2.5	HLD, centroid, LCT, virtual tree
		lazy seqtree, Li Chao, treap, persistent, rollback

Layout that works under stress

- **Landscape, 2–3 columns**, monospace 7–8 pt, syntax highlighting off or greyscale
- One snippet never crosses a column break
- Header per snippet: **name · complexity · usage · hash**
- Short identifiers, no blank lines, comments only for *preconditions*
- Table of contents on page 1 with page numbers; sections in a **fixed order** the team memorises
- Build from source files with a script (LaTeX like KACTL, or `pandoc` / `enscript`) — never edit the PDF

Snippet header example

```
3D convex hull                                  O(n^2)                  hash: 7c1e2a
Usage: hull3d(pts) -> CCW (from outside) triangles; {} if coplanar.
Preconditions: integer coords |x| <= 1e6 (uses __int128 in vol()).
Merge coplanar triangles yourself if faces must be polygons.
```

- *Preconditions* are where WF bugs live: coordinate bounds, 0- vs 1-indexing, multi-edges, n = 1
- The **hash** lets the typist check a retyped snippet in 5 seconds

Hashing retyped code

```
#!/usr/bin/env bash
# Hash a snippet ignoring whitespace and comments.
cpp -dD -P -fpreprocessed | tr -d '[:space:]' | md5sum | cut -c-6
```

- Print the hash of every snippet (and of every ~10-line block in long ones)
- After typing: `./hash.sh < hull.cpp` → compare with the paper
- Mismatch ⇒ typo; bisect by block hashes
- Costs 1 minute; saves the 40-minute "the library must be wrong" hunt

Testing every snippet

Kind	Test
Algorithm with brute force	randomised stress test, tiny + degenerate inputs
Data structure	random operations vs naive arrays
Library-Checker-style	submit to an online judge (Library Checker, CSES, CF)
Geometry	structural checkers + integer coordinates in [−3, 3]
Numeric	compare against `long double` / exact rational version

Rule: a snippet without a recorded test (file + date + result) **does not get printed**.

Verification map (excerpt)

Snippet	Verified on
NTT, poly inv/log/exp	Library Checker *Convolution*, *Inv/Log/Exp of Formal Power Series*
Berlekamp–Massey	Library Checker *Find Linear Recurrence*
Hungarian	Library Checker *Assignment Problem*
Link-cut tree	Library Checker *Dynamic Tree Vertex Add Path Sum*
SA + LCP, Z	Library Checker *Suffix Array*, *Number of Substrings*, *Z Algorithm*
Lucy / Du	Library Checker *Counting Primes*, *Sum of Totient Function*
3D hull	random stress (closed surface + all points inside + Euler)
Aliens, slope trick	stress against $O(nk)$ / grid brute force (Lecture 9)

Keeping the TRD alive

- One **git repository**: `src/<section>/<snippet>.cpp`, `tests/<snippet>_test.cpp`, `build.sh`
- CI (or a pre-print script): compile every snippet with `-Wall -Wextra -fsanitize=address,undefined`, run every test
- Changelog: who changed what, after which contest
- **Freeze** the TRD 2 weeks before the Finals; print 2 test copies and use them in simulations
- After each simulation: add / cut / fix — the TRD evolves with your mistakes

The template and scripts page

```cpp
#include <bits/stdc++.h>
using namespace std;
#define rep(i, a, b) for (int i = (a); i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
int main() {
    cin.tie(0)->sync_with_stdio(0);
    cin.exceptions(cin.failbit);       // crash loudly on malformed input
}
```

Plus: compile alias (`-O2 -Wall -Wextra -fsanitize=address,undefined -D_GLIBCXX_DEBUG`), `stress.sh` , `hash.sh` .

The 5-hour simulation: setup

- Real conditions: **one computer**, three people, 5 hours, printed TRD only, no internet except the judge
- Use a past WF set *nobody has seen* — or the original 12-problem mock set in the notes
- Replicate the announced contest environment: OS image, editor, compiler flags
- A **coach/observer** logs timestamps: who is at the keyboard, every submission, every switch
- Scoreboard: freeze in the last hour, as at WF

Timeline (plan, written before the start)

Time	Activity
0:00–0:20	split the set 4/4/4; each reads, tags (topic, est. difficulty, est. code length)
0:00	keyboard: type template + scripts, hash-check them
0:20	2-min stand-up: easiest problem per person; first coder starts
0:20–1:30	easy problems; watch the scoreboard for unexpected easy ones
1:30–3:00	mediums; pair-think while third codes
3:00	**checkpoint**: rank remaining problems by P(solve) / time
4:00	freeze — commit to ≤ 2 problems, no new reading

Keyboard discipline

- Nobody touches the keyboard without a **complete plan on paper** (data structure, complexity, edge cases)
- WA ⇒ **print** and leave the keyboard; debug on paper while someone else codes
- 10-minute rule: stuck at the keyboard for 10 min ⇒ swap
- Before submitting: samples, a max-size test for time, n = 1 / empty / all equal
- Long implementations (LCT, 3D hull, blossom) are typed from the TRD **with hash checks**

Communication protocol

- Short, structured: "B: necklace Burnside, n up to 10^{12}, 15 lines, ready to code"
- Shared paper sheet per problem: status, idea, owner, attempts
- Ask for a **second opinion** before the first submission of anything ≥ medium
- Do not interrupt the coder except for: "stop, the idea is wrong"
- Name the problem a teammate is stuck on out loud at checkpoints — fresh eyes solve hard problems

The original mock set (notes §5)

	Problem	Topic	Difficulty
A	Lighthouse Keepers	greedy interval cover	easy
B	Proper Necklaces	Burnside + φ	medium
C	Palindrome Diary	eertree	easy–medium
D	Smooth Skyline	slope trick, sliding window	medium
E	Crystal Facets	3D hull + coplanar merge	medium–hard
F	Festival Budget	Aliens + CHT	hard
G	Visible Orchard	Möbius + Du's sieve	hard
H	Skill Teams	SOS + inclusion–exclusion	medium
I	Shifting Alliances	link-cut tree	hard

Expected outcomes

- Gold-level team: 8–10 problems; silver/bronze: 6–8; honourable mention: 4–5
- Easy core (A, C, H, K, B): everybody should solve these in < 2.5 h total
- Discriminators: D, E, F, G — technique known, implementation decides
- Hard tail (I, J, L): **one** of them is a good result; choosing which is the test
- Optimal reading of the set *is* part of the score: did you identify C and H as easy within 20 minutes?

Endgame decisions

- At 4:00 for each open problem estimate: remaining minutes to AC, probability
- Expected gain ≈ P(AC) — prefer the problem you have **already debugged** over a fresh one
- Penalty matters only for ties; never skip a likely AC to save penalty
- Last 15 minutes: submit with extra checks on (assertions removed? output format?)
- After the contest: **nobody leaves** before the post-mortem notes are written

Post-mortem: per-problem log

Field	Example
Read at / by	0:07 / Ana
First correct idea at	0:41
Coding start / end	1:05 / 1:32
Submissions	WA (1:33), TLE (1:51), AC (2:06)
Root cause of each rejection	off-by-one in window; `map` instead of array
Minutes lost	33
Detectable earlier by	max test; edge-case checklist

Post-mortem: team questions

1. Which problem did we misjudge most (difficulty or topic)?
2. Where did the keyboard sit idle? Where was it contended?
3. Which TRD snippet was missing, wrong, or slow to find?
4. Which bug class appeared (overflow, indexing, precision, I/O)?
5. One thing to **start**, one to **stop**, one to **keep**

Output: ≤ 5 action items with owners and deadlines (e.g. "add polygon-cut to TRD — Ben — Friday").

Simulation cadence before WF

- Monthly: full 5-hour simulation + same-day post-mortem
- Weekly: 3-hour set with team rules, alternating keyboard roles
- Daily (individual): one hard upsolve from the track topics
- Two weeks out: TRD frozen; simulations use the printed copy only
- Final week: rest, short warm-ups, environment rehearsal

Common pitfalls

- TRD as a textbook: pages of theory nobody reads during the contest
- Untested snippets ("we copied it from a blog the night before")
- Different code styles per member ⇒ nobody can debug anyone else's code
- Simulation without a written plan ⇒ no way to measure deviations
- Post-mortem that blames people instead of processes
- Changing the TRD in the final week

Practice

- Build v1 of the TRD from Lectures 2–9 snippets; run `build.sh`; print it
- Verify ≥ 10 snippets on Library Checker problems listed in the notes
- Run the **mock set** (notes §5) as a 5-hour simulation with an observer
- Then: past WF sets on Kattis / the ICPC problem archive (one per month, unseen)
- Write the post-mortem with the template within 1 hour of the end

Summary

- TRD: only tested, frequently-needed, hard-to-recall code; 25 pages with a fixed order and hashes
- Every snippet has a test and a verification record; build from source in CI
- Simulate the full 5 hours with a written plan, an observer and a frozen scoreboard
- The post-mortem converts each lost minute into an action item

Good luck at the Finals.